

3. Defining an Event-Driven Microservice and Its Boundaries

Hugo Filipe Oliveira Rocha¹

(1) Ermesinde, Portugal

This chapter covers:

- Understanding the structure of an event-driven microservice, associated message patterns, and common topologies
- Comprehend DDD (domain-driven design), how to apply it, and its concepts
- Using DDD to extract the domain's bounded contexts and leverage the services' boundaries
- How to avoid common pitfalls when designing the aggregate size and understanding its impact on downstream services
- What different characteristics to weight when organizing service boundaries
- Understanding the difference and the impacts of request-driven and event-driven services
- How to decide between adding functionality to an existing service or creating a new one

It's funny how sometimes computer science feels more art than science. Perhaps not more, but it often has a soft brush of artistic inspiration mixed with it. We often fail to see the practical value in learning all the computer science fundamentals in class or learn about new patterns in books or articles. Usually, we use them as building blocks or extract their underlying ideas to combine and adapt them as solutions to unique problems. They often serve as an inspiration to other solutions in different contexts. Sometimes creativity plays a role, and your personal style gets associated with it. How often have we looked at someone's code and underline the invisible traces that define a coworker's style? We can feel the

style the same way we would if we read several poems of a unique author – who is probably sitting next to you, trying to exit Vim.

When defining domains and boundaries, it can also get fuzzy and often feels more like art than science. There isn't a magical formula we can apply and obtain a result, but there are approaches that can help us extract business domains and decide what is the best approach. This chapter approaches those strategies and helps us understand the application's domain and translate it into a logical model.

In Chapter [2](#), we discussed how to extract functionality from a monolith to a new service. The different types of services and topologies we can build in an event-driven architecture are the building blocks we can use to evolve that architecture. Understanding them is paramount to maintaining and building these architectures.

As we evolve the architecture and depend on several different components, the need arises to define the services' boundaries, how they interact with each other, and what domain they manage. Different organizations have different priorities; often deciding the adequate boundaries is a tradeoff between the different approaches. We will discuss the various considerations when defining boundaries in Section [3.2](#). DDD is an approach we can use to understand and extract the domain concepts. When modeling complex domains, the entity's size (the aggregate in DDD terms) is often hard to measure. In an event-driven architecture, how we design it can have repercussions to the downstream consumers; this impact is detailed in Section [3.4](#).

Adding new functionality usually falls into one of two choices, creating a new service or adding functionality to existing services. Sometimes the choice isn't clear; the high decoupled nature of event-driven architectures promotes the birth and decommission of components. However, continuously adding services further increases the system's complexity. Section [3.6](#) further details these tradeoffs.

3.1 Building Event-Driven Microservices

This section will discuss the typical architecture of an event-driven microservice and its standard modules. We will discuss the typical messaging entities and messaging patterns that we will use in the rest of the book as building blocks in the architecture. At the end of this section, we will discuss common pitfalls, their impacts, and how to avoid them.

In Chapter [2](#), we moved functionality from a monolith to event-driven services. We represented it with a square, but what is inside that square? Each service is a single-process application, and inside it, the code is organized typically according to an architecture. Usually, we divide this architecture into layers. One of our missions is to model business concepts and processes into code. As the business grows, the application also grows in complexity. Organizing the application with logical layering is one way to manage that complexity.

Dividing an application into layers according to its responsibilities aligns with the separation of concerns principle. It's also a way to help developers understand the application and easily find functionality. Maintaining the same architecture through several microservices minimizes the overhead of understanding an application and implementing new functionality. If someone never worked in one service and now has to change it, although the domain might be different, the application's organization is the same, so it is easier to understand.

We can use a layered architecture to encapsulate functionality inside each layer, promoting the code to change without affecting other layers. The functionality can also be reused throughout the application standardizing common functionality in a single implementation. We can also use a layered architecture to limit coupling and restrict how layers communicate with each other. They also become useful to develop unit tests since the separation of concerns promotes simple single-purpose tests.

There are several types of layered architectures. Due to the natural decoupling of event-driven services, we don't need to use the same in every service, as we don't need to use the same technology in every service either. However, it is advisable to choose one and maintain the consistency between services. As we will discuss in Chapter [10](#), if other teams need to change services in the ownership of other teams, the development is

more straightforward if every service has similar architecture. If the structure of applications is familiar, the focus of discussion shifts from technical details to the domain implementation, which is often more valuable.

3.1.1 N-Tier Architectures

The most common architectures feature a separation of the application code into several layers. There are several variations; the number of layers depends on the application's needs (thus the name N-tier). Figure [3-1](#) illustrates a typical three-tier architecture.

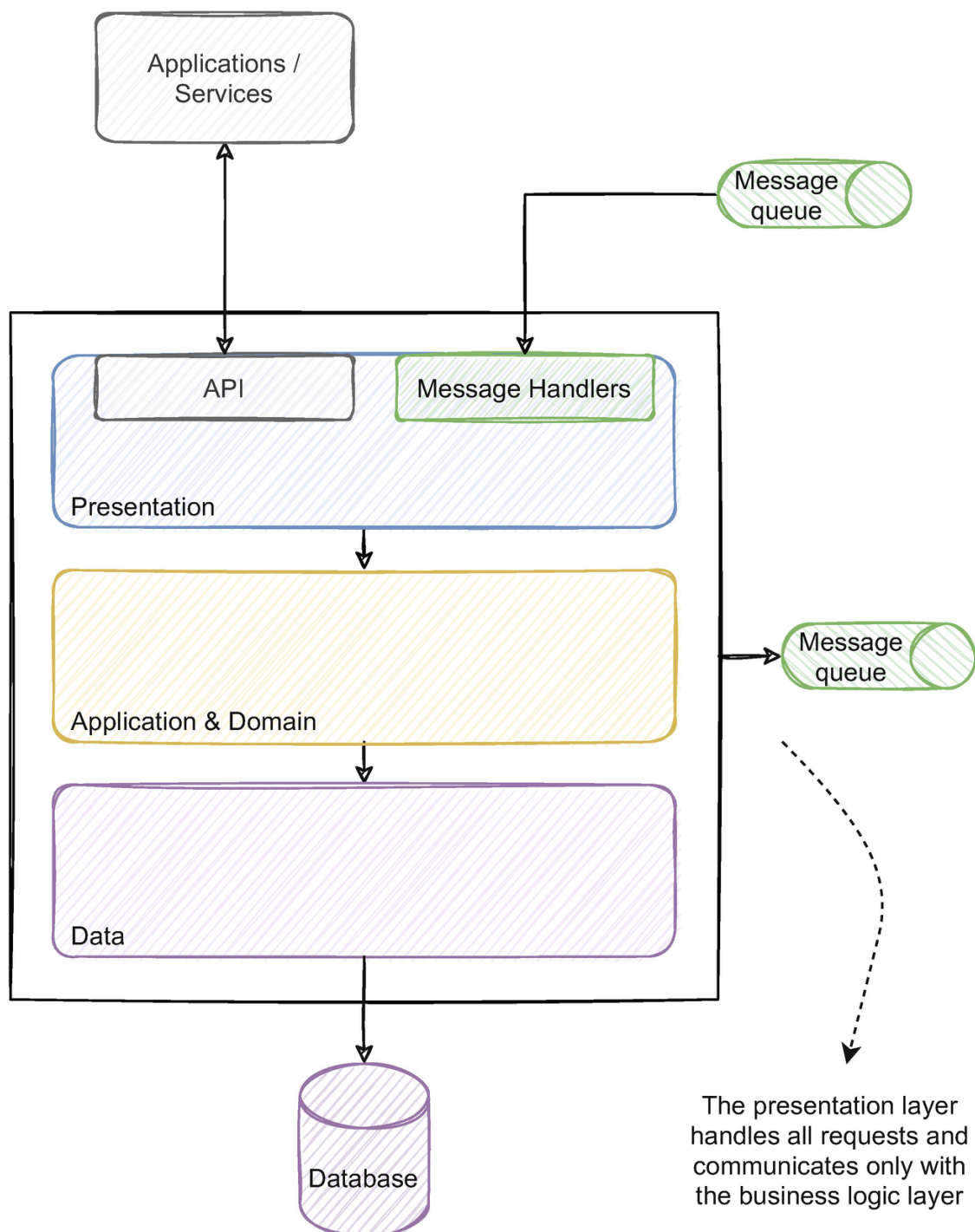


Figure 3-1 Three-tier application architecture with three layers: presentation, application and domain, and data

In this architecture, the presentation receives all external requests; if the microservice has a UI, we also place it in this layer. Microservices without UI expose an API that other applications and services use to access the data. Typical event-driven microservices communicate through events. Event handlers receive the events and call the other layers to process their business logic. The presentation layer only interacts with the application and domain layer, which has all the business logic. The application and domain layer manages the state through the data layer, which has all the data access objects and the database technical implementation.

The most important part is the business logic that resides in the application and domain layer. One of the drawbacks of this architecture is the direct dependency of the application and domain layer to the data layer. Often this means the business logic depends on data access, which shouldn't be relevant for the domain logic itself. The database implementation details often influence the implemented business logic. The existence of a database also often becomes a required prerequisite to the business logic. Testing might even become an issue due to database coupling. Often to test the business logic, we require a staging database; we can avoid that with dependency injection, which often leads to a different kind of architecture that we detail in the following.

3.1.2 Clean Architecture

An alternative to N-tier architecture is Bob Martin's clean architecture ¹. As we mention in the following, if we adopt the dependency inversion principle (the I in SOLID²) and depend upon abstractions rather than implementations, we often reach this architecture. It is also designed to build the domain logic in the core of the application. Domain logic shouldn't depend on or reference anything else; the order management business logic doesn't need to know we use SQL, for example.

Other architectures are typically designed upon the same practical concept the clean architecture translates. The hexagonal architecture³ (also named ports and adapters) or

the onion architecture⁴ provides a similar approach based on the same principles. They strive to provide a clean separation of concerns through the segregation of the software into layers. They also emphasize the business logic as the focus of the application. The business logic also has no dependencies or references, the outer layer references the domain models, but the domain models don't reference anything. Figure 3-2 illustrates a typical microservice with the clean architecture.

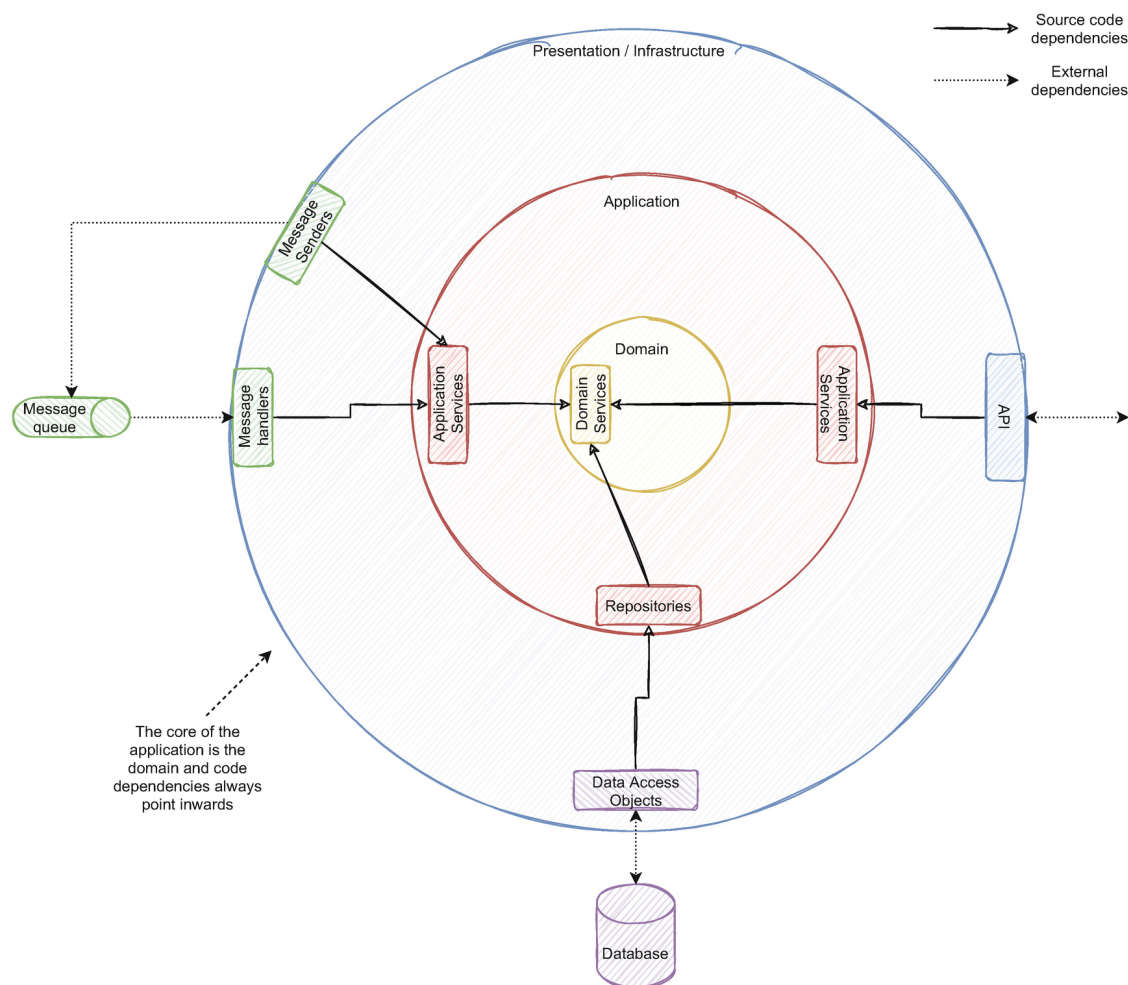


Figure 3-2 An application using clean architecture; the code dependencies are always inward. The domain layer is the core of the architecture and hasn't any references to other layers

We only reference external dependencies in the outer layer. Code in each layer can solely depend on references to the inner layers. The core layer holds the domain logic and domain entities and can't reference anything in the outer layers. This way, anything in the outer layer can't impact the inner layers. Outside layers typically have concrete implementations, while the inner layers grow more abstract as we advance to the core. Figure 3-2 only shows three layers, but it is just an example; an application can have as many layers as needed. The application layer, which holds the application business logic, can have an additional layer with in-

terface adapters in charge of adapting data between the presentation layer and application logic.

Having the external dependencies on the outer layer means we reference every tool, driver, or framework in that layer. Having external dependencies referenced only on the outer layer promotes the separation of the application and domain logic from external tools. Often, the limitations of these frameworks or technologies entangle themselves on the application's logic. Moving them to the outer layer promotes the modeling and evolution of the logic without the influence of outside tools. We often hear “If we need to exchange databases, we can do it without affecting the inner logic.” Separation of concerns is an essential coding principle, but how often do we exchange databases anyway? Database constraints tend to leak toward application logic (e.g., managing transactions between several models) or framework updates that span all applications. The outer layer helps prevent these common issues.

Also, domain logic shouldn't know anything about what we implemented in the outer layers. The independence from outside references (like the data reference in the N-tier architecture) helps the business rules evolve as needed and reflect what the business intends. Changes in how we authenticate users, how we publish events, or how we log changes shouldn't affect the business rules. Having this kind of separation and isolation, they won't. Figure 3-3 illustrates two services using this architecture, interacting through a message queue.

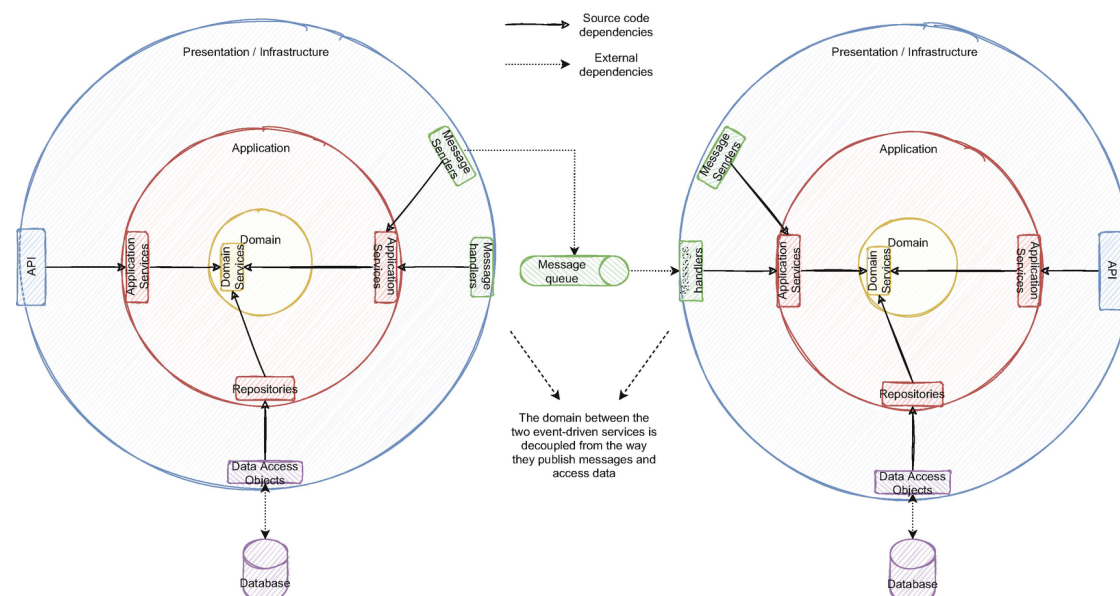


Figure 3-3 Two event-driven services with clean architectures interacting through a message queue. The domain is isolated from the way the applications send events and access data

If, for example, the messaging framework we are using in both services becomes deprecated, upgrading it will only affect the outer layer. It eases the replacement of external dependencies, and it also promotes single-purpose unit tests in each layer.

We can apply to each microservice the clean architecture. Still, as we add more functionalities and new services, we might create smaller services that aren't that rich in domain or application logic. First, as we mentioned in Chapter [2](#), it is advisable to develop services that encapsulate a subset of the system but a more extensive and autonomous domain. Creating larger coarse-grained services, in the beginning, will naturally contain the expansion of the number of services to a sustainable pace. They will also depend less on external services that are easier to manage while building the architecture. It will reduce the number of remote calls each service has to do and lessen the impact of weak consistency since each service will manage its domain locally. As we mentioned in Chapter [2](#), we should gradually build the system, adding coarse-grained services first and then, when needed, advancing to fine-grained services with lesser responsibilities.

As we evolve the architecture and create more fine-grained services, we might have smaller services that own part of the overall architecture's responsibilities. Figure [3-4](#) illustrates an example where there is a service to manage the domain logic.

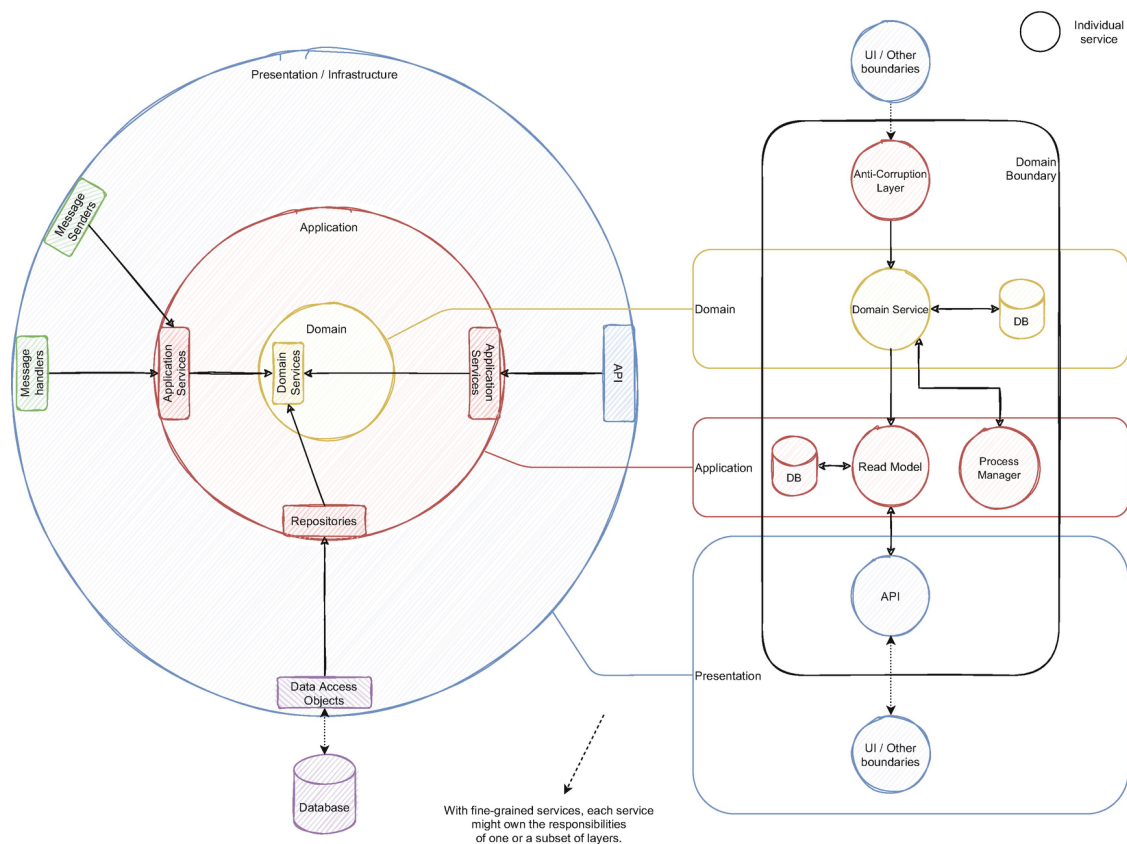


Figure 3-4 As we create more fine-grained services, a subset of services inside a domain boundary might map with one or a subset of clean architecture layers

Other services like the read model and process manager operate the application logic, and the anti-corruption layer works as an adapter between external and internal information. The API service exposes the boundary's information.

There are pitfalls associated with having fine-grained services that we need to avoid. Such a small scope often increases the risk of having lock-step releases, where a functionality spans several services and requires coordination to deploy them in the live environment. Microservices should be autonomous, and a release of one service shouldn't affect others. Also, services with a small scope will rapidly increase the number of services, increasing the system's complexity, and exponentiate the challenges of distributed systems discussed in Chapter 1. We can contain those challenges by organizing the services into boundaries, as illustrated in Figure 3-4. However, creating coarse-grained services first and using fine-grained services later is a better approach to decompose the system incrementally.

3.1.3 Event-Driven Microservices: Durable vs. Ephemeral Message Brokers and GDPR

The availability of messages after consumption and the underlying messaging semantics are the foundation for data streaming and provide a novel way to share data. Message brokers can have durable or ephemeral messages depending on the broker we choose. Typical messaging has ephemeral messages; however, having durable messages potentiates the useful use cases we mentioned in Chapter 1 and which we will detail throughout the book. In this section, we will discuss the two approaches of using durable or ephemeral message brokers.

Traditional message brokers are built to have short-lived messaging; published messages typically are removed once consumed. A famous example is RabbitMQ that is designed for destructive consuming semantics; once the consumer acknowledges the message, the broker removes the message from the queue. RabbitMQ also features⁵ an append-only log data structure and will provide non-destructive messages besides the current destructive semantics.

With the rise of the Internet and continuous data growth, event streaming became increasingly relevant, along with durable message brokers. Durable messages remain readily available even after processed by the consumers. Streaming is about enabling a distributed and ordered flow of events. Durable messaging allows the stream to be processed as many times as needed. A famous example is Kafka, which is a distributed event streaming platform and provides a persisted event log. Like the event stream in Kafka, durable events are always available even after an application has consumed them. Other consumers can access the event stream, or existing consumers can reprocess the same messages if needed.

As with many of the technologies we use, the decision is about weighing tradeoffs. Using durable or ephemeral message brokers in every use case might not always be the right decision; it depends on what we need them for and its use case. Ephemeral message brokers like RabbitMQ have a high focus on point-to-point communications and routing messages to specific channels. Short-lived messaging is usually best for con-

sumers who can process the messages fast and don't retain the broker's messages for long periods. These brokers often struggle when large numbers of messages pile up in the queues and are best used for instantaneous low-scale communications. In our experience using RabbitMQ for years in production on a high-throughput platform, we struggled sparingly with message load peaks that affected the whole cluster. When the entire cluster struggles, it will likely affect unrelated services and have daunting impacts, even imperiling the whole platform. Looking back, the use case for RabbitMQ might not be the best in that situation because we didn't need short-lived messaging.

For event-driven architectures, durable message brokers have high synergy with the event-driven mindset. Exposing the application state and maintaining it for any consumer in the form of events potentiate powerful possibilities and provide a sustainable way to share data throughout the company. Brokers like Kafka can also horizontally scale and manage vast amounts of messages, often achieving⁶ better throughput and performance than typical short-lived message brokers. However, non-destructive message semantics usually has an associated cost and overhead of adding new brokers due to the need to replicate the data between nodes. Overall, event-driven architectures greatly benefit from durable message brokers, and many use cases become notoriously simple by having one. As we will see, many of the patterns we discuss in this book benefit from durable message brokers.

The Right to Be Forgotten and Other Security Concerns

Durable message brokers imply a challenging concern about securing the data. There is a lot of value in every event being available in the broker for any consumer even after consumption and requires dedicated security strategies. Typically, data that is no longer needed often is a security risk. Durable event brokers also raise several challenges guaranteeing compliance with regulations like GDPR (general data protection regulation). Compliance with the right to erasure or right to be forgotten is particularly challenging in an event-driven mindset. As we discussed, durable event brokers persist the event stream and provide access to it to any consumer. When a user wants to be removed from the system, we also store data associated with the user in the message broker, and we

need a way to destroy that data. Ephemeral message brokers naturally comply with these requirements since they remove messages after consumption.

Deleting an entity in an event-driven mindset is often accomplished by an event signaling the deletion of an entity (e.g., if the entity is an order, the service deleting the order would publish an order deleted event). This concept is often referred to as a tombstone; Cassandra, a popular NoSQL database, for example, when deleting a record, doesn't immediately purge it from the disk; instead, it writes a value signaling the record was deleted.

Theoretically, event streams reflect what happened, the history of the entity with all its evolution; every event is the reflection of a unique operation and shouldn't be deleted, the same way we can't delete history and what happened in times past. Many durable event brokers don't have straightforward, built-in ways to delete specific events and often require ad hoc solutions.

For example, the right to be forgotten introduces a unique challenge; if we shouldn't delete events, how can we comply with that requirement? Ephemeral message brokers deal with this naturally since the broker effectively deletes messages after consumption. Since the broker deletes them, it organically complies with the requirement, and we do not need a strategy to deal with data deletion.

An approach to solve this challenge is using crypto shredding. Crypto shredding is a strategy to destroy data by losing the encryption keys needed to understand the data. Encrypted data can only be understood by decrypting it with the corresponding key. By deleting or overriding the key, the data is inoperable. Encryption is also a common requirement and needed to achieve compliance with GDPR, having a synergy between encrypting the events and using crypto shredding to destroy data. Figure [3-5](#) illustrates how we could apply crypto shredding to event-driven services.

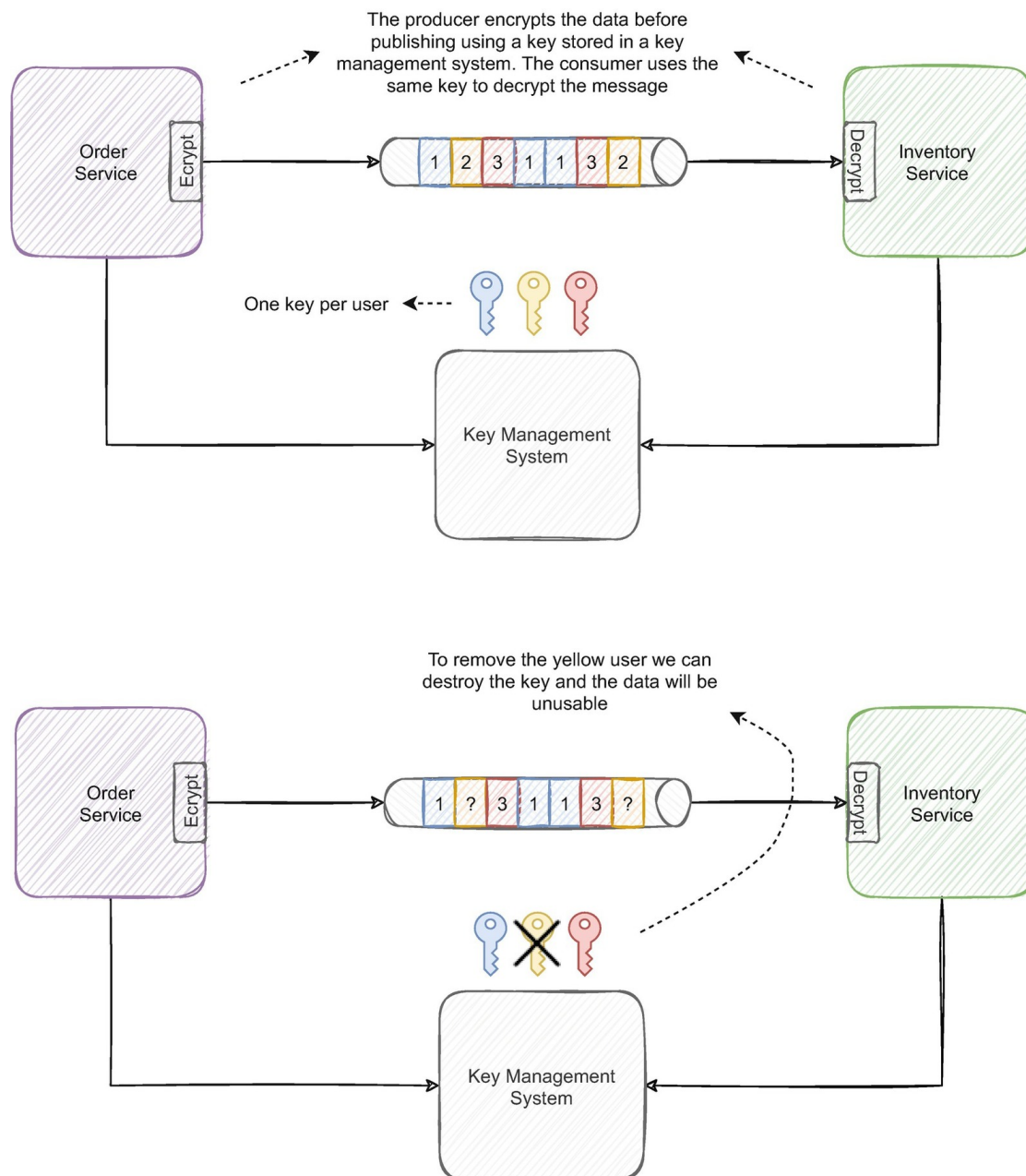


Figure 3-5 Example of using crypto shredding to destroy messages associated with a user

The order service encrypts the data using a key in a key management system. Each user has a different key, and we encrypt each user's events with the respective key. The inventory service to use the data has to decrypt the events by using the same key used to encrypt. If a user wants to be removed from the system, we can destroy the key associated with that user and the data will be unusable.

Applying this strategy to a complex architecture with several microservices can become challenging since each one of them would have to implement this strategy. Often, not all services need encryption and deal with sensitive data. Identifying and isolating the sensible data to the services that need them and applying this strategy only to those services

can also be a good approach. There is also the option to use this strategy at the infrastructure level avoiding the implementation effort in all services. We can proxy the requests to a common module which has the responsibility to encrypt the data before publishing them to the broker. David Jacot details this approach in an interesting presentation⁷ by shipping containers with that additional module (known as a sidecar pattern) using Kafka.

3.1.4 Event-Driven Message Types

Traditional messaging uses messages somewhat indifferently; they are just a means to propagate data. In event-driven architectures however, there are different types of messages that have different purposes and different meanings. In this subsection, we will discuss the different types and in which situations they should be used.

Messages are usually composed of a header and a body. We can use the header to pass additional information common to all messages, much like HTTP headers. A typical example is the correlation id, which we often use to link several messages related to the same entity. For example, if we create a product and then edit its brand and category, each of the changes would publish a different event (e.g., the generated events correspondingly: `ProductCreated`, `ProductBrandChanged`, `ProductCategoryChanged`); all three events could have a header with the id of the product signaling they are all related to the same product. Although there isn't a formal definition of what headers should a message have, we benefit from having a predefined set for the entire system. The correlation id we discussed before is a good example, but having other message headers like a message id that uniquely identifies each message can be useful for debugging purposes. The headers' information must be chosen carefully in order to avoid flowing information that will have logic associated on the consumer side, like specific flags for specific processes. Often this kind of information can be forgotten, leading to errors on the consumption process or even assuming default values for the headers; Chapter 8 will further detail this subject. Having an event version or timestamp is also helpful for managing idempotency, as we will discuss in Chapters 5 and 6.

The message body has the full information we want to publish. Messages can be commands, events, or documents. Queries are also a common concept, and typically, they aren't a message but a common concept in event-driven architectures.

- **Commands** are orders to perform a given action. We should name them with a verb in the imperative form, for example, `CreateOrderCommand`. A command is a request to a given service to perform an action and thus can be rejected or fail a validation. We can change aggregates by sending a command to perform a given action in that aggregate and often reflect a user's action. Typically, commands affect one service and a specific domain; they usually aren't published to several subscribers, only one. Although often commands are messages, a command can also be an HTTP request if the service receives changes through an API instead of a message broker.
- **Events** notify something that has changed in a given domain or aggregate. They are named in the past participle verb and inform that something has happened, for example, `OrderCreatedEvent`. They are facts and, unlike commands, aren't liable to be rejected; they are something that already happened. Events are the most common block of event-driven architectures and are used to pass information and signal relevant changes throughout the architecture's components. They are often published to several consumers and can accommodate new consumers in the future, unlike commands that are related to a single system.
- **Documents** are much like events; the service publishes them when a given aggregate or entity changes, but they contain all the entity's information, unlike events which typically only have the information related to the change that originated the event. Although they often are triggered by changes in the aggregate, they often don't give information about what change triggered the document, unless we specifically add it to the document. If someone changed an order address, the generated event could be `OrderAddressChanged` and contain the information about the new order's address. The same change could trigger a document, for example, `OrderDocument`, which would have all the order information; each receiver would have to interpret it in the way that made sense to that service.
- **Queries** are requests of information issued to a given system to obtain the service's data. Typically, they aren't messages and are often syn-

chronous requests like an HTTP request. They also don't change state; they simply request information.

Figure 3-6 illustrates an example of an interaction between these entities.

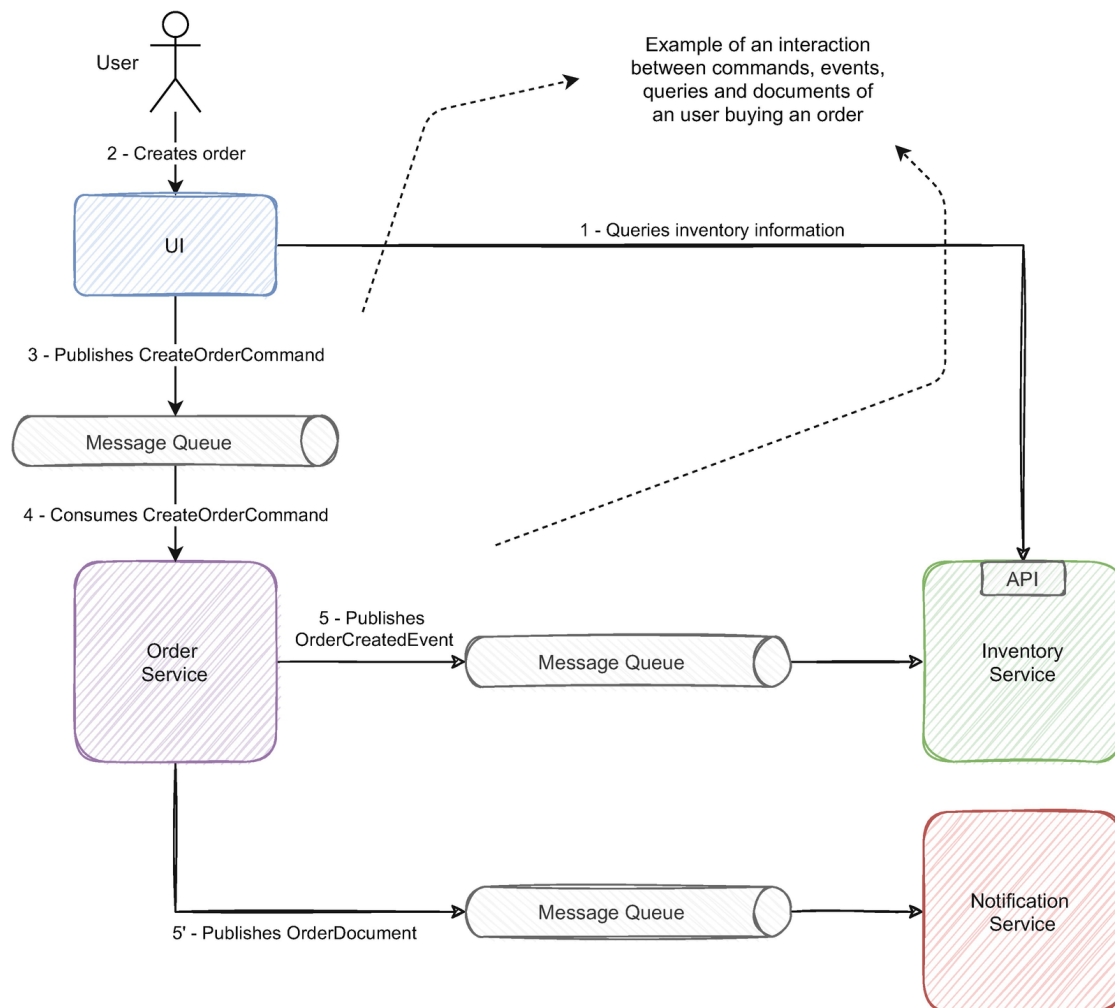


Figure 3-6 The interaction between commands, events, documents, and queries

A user placing an order would query the product's stock to the inventory service and then trigger a `CreateOrderCommand` by submitting an order that would be consumed by the order service. Once the service creates the entity, it publishes a `CreateOrderEvent` and an `OrderDocument` which are consumed by the inventory service and the notification service, respectively. Commands, events, documents, and queries are the foundation for the patterns and topologies we discuss further in this chapter.

3.1.5 Event-Driven Microservices: When to Use Documents over Events

In Subsection [3.1.2](#), we discussed we could use either events or documents to notify changes in entities. Choosing to publish an event or a document can be debatable, and they often accomplish the same goal. However, they are fundamentally different, and we should use them in specific use cases. In this subsection, we will discuss several topics that will help us decide where to use documents or events.

Events represent a specific change and have domain value by themselves since they represent something the user did or a change in the domain. Documents just inform the entity's latest state, so they lose the domain value carried by the event's meaning. For example, in Listing [3-1](#), there is an example of a partial event and a document representing the change in an order's address.

```
1  OrderAddressChanged
2  {
3      OrderId: 15251212,
4      Address: "9980 Rock Maple Street",
5      UserId: 12162
6  }
7
8  OrderDocument
9  {
10     OrderId: 15251212,
11     Address: "9980 Rock Maple Street",
12     OrderLines: [
13         {
14             ProductId: 1147421,
15             Quantity: 1,
16         }
17     ],
18     OrderedAt: "2021-01-23T18:25:43.511Z",
19     UserId: 12168
20 }
```

Listing 3-1 OrderAddressChanged vs. OrderDocument

If the receiving system needs to react only to changes in the address, it is more beneficial to use the event. For example, if a billing service had to update the address in the invoice, whenever the user changes the order's address, it would be easier to handle the OrderAddressChange. Using the OrderDocument would need to save the orders internally after each event or request the order from the order service to understand the address had changed.

However, documents often simplify consumers that need more information from the entity than an event would make available. For example, if we need to notify the user with all the order information every time an order changes, we would benefit from using the document in Listing [3-1](#) since all the information is available in the message. It would prevent the service to request or store the additional information that wasn't available in the event to send the notification; this would greatly simplify the consumer. With partial events, it also becomes complex to maintain all the entity's data. For example, if the order entity would grow and have more and more properties, it would probably imply that we needed to create more and more partial events. If the consumer needs to maintain all the order data, it is more complex to handle several different events than one with all the information. Another advantage of using documents is that if the user changed the address and quantity simultaneously, that would mean two different partial events. In contrast, if we use the document, we only need to publish one.

An exciting approach with Kafka is to use compacted topics. Compacted topics in Kafka have a periodic mechanism to remove messages with an older version for a given id, maintaining only the latest ones. So if we combine compacted topics with documents, we have a message queue with all the orders' information, much like a database table but in the form of events in a medium oriented to be shared with other consumers. In the case of the example in Listing [3-1](#), we could use the order id and publish the OrderDocument for each order; this way, the topic would have all the orders' information, much like the lines on a SQL table but in the form of messages ready to be shared. Consumers can plug to the topic and stream the data directly to their application. This way, we provide a way for applications to receive the data without impacting the origin service and its database. Since the broker is durable, we can read the topic from the beginning if we need to replay the data or new consumers need initialization.

Overall, it becomes simpler to use documents when we need a large part of the entity's data. However, we lose the meaning and the inherent change behind the event; if the consumer needs that meaning to process its logic, it will have to infer the meaning internally. In those cases, we are

better off using partial events. It is often more useful to use events that reflect the user's intent, and this should be our go-to approach. But we always need to have the event consumer's needs in mind and adapt accordingly. In some use cases like we mentioned before, documents are useful; we will further detail this concept in Chapter 8.

3.1.6 Common Event-Driven Messaging Patterns

In Subsection 3.1.2, we discussed the types of messages and how they differ from each other. This subsection will discuss how they are typically used and the different patterns they are organized into.

As we mentioned in Chapter 1, event-driven architectures are composed of services that publish events to event brokers. Other services react to those events to accomplish their business rules. The services are built to be horizontally scalable, and several instances of the same service can consume from the same queue, having the load distributed between all of them.

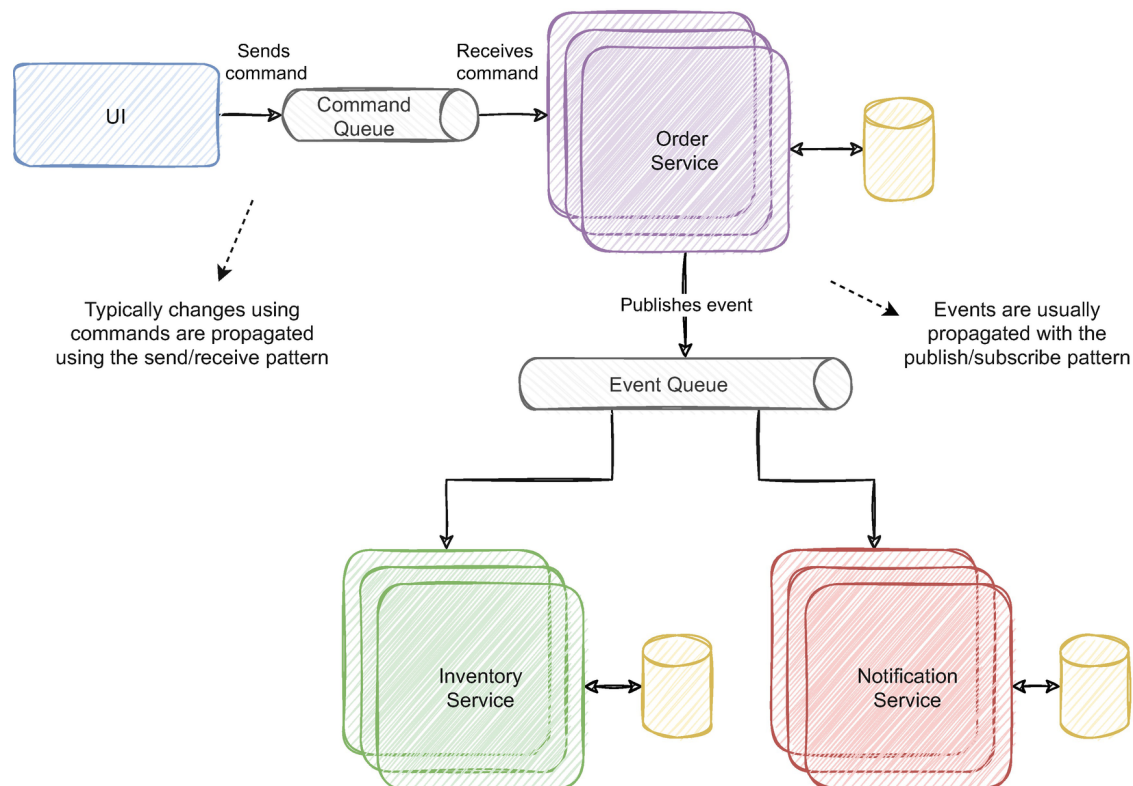


Figure 3-7 Typical messaging patterns used in an event-driven architecture

Figure 3-7 illustrates a typical architecture, much like the examples we discussed until now, using an order, an inventory, and a notification service. Changes are triggered by the UI, where the user changes, for exam-

ple, its order information. To change the order, the UI sends a command to the order service, which publishes an event that is handled by any application that needs order events. Notice, however, several squares are representing the services, meaning each service can have multiple instances.

Send/Receive Pattern

The interaction between the UI and the order service is a send/receive pattern. Typically, commands are requests to change a given entity or aggregate information and have a very specific receiver. The request to change the data is only relevant for the order service; no other service needs or should receive that command. Send/receive is typically a point-to-point communication between two services with a specific purpose, often a request to do a given action on that service.

It is essential to distinguish the publication pattern along with the types of events. Commands are typically send/receive; events are typically publish/subscribe. Commands often have a very specific purpose and have a close relation with the domain they are changing; it is important to guarantee only the desired service reacts to the command.

Publish/Subscribe Pattern

The interaction between the order service and the inventory and notification service is a publish/subscribe pattern. The order service publishes changes that happened in its domain, and interested services subscribe to those changes and process them. As the previous pattern, typically they are fire-and-forget; the originating service only guarantees that the message was published to the broker.

Multiple services can handle the events and will process them at different rates. Each service might have several instances, as depicted in Figure [3-7](#), and each instance will receive various events and process them at different rates in parallel. This is the basis of horizontal scalability, which also introduces complex challenges on how to handle the concurrency between the several instances, out-of-order events, and eventual consistency associated with it, which we will discuss further in this book.

Request/Response Pattern

The interaction between the UI and the order service is intrinsically asynchronous due to happening through a message queue. The UI also doesn't know when the changes took place. The UI can notify the user the changes took place by handling the corresponding event. Once the service handles the command and applies the changes, it will publish an event signaling those changes. The command and the event can be correlated by an id which can be used by the UI to understand that the requested changes were applied. This way, the UI can be a publisher of commands and a receiver of events, implementing the request/response pattern. The changes can be linked together by a correlation id that would be published in the command and transmitted in the event. Figure 3-8 illustrates how the patterns we discussed before can be applied conceptually.

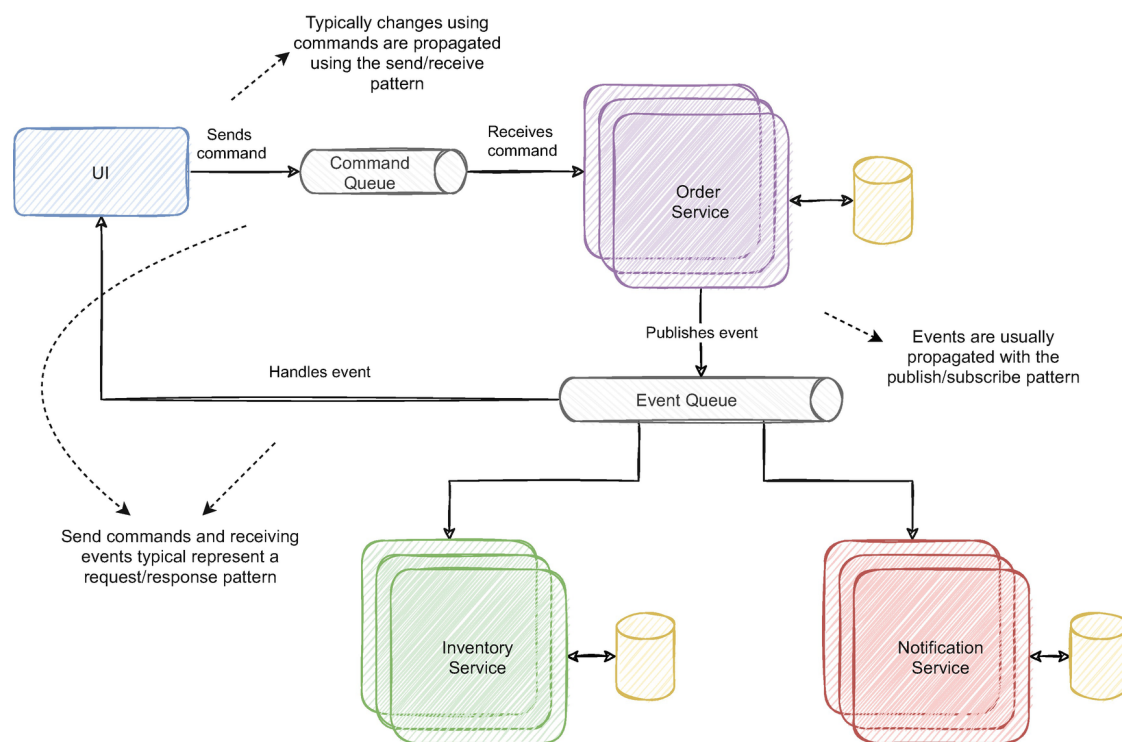


Figure 3-8 Simple examples of the patterns we discussed before using the same example as before

The UI implements both send/receive and request/response patterns. It sends a command which is received by the order service and handles the event generated by that command. The publish/subscribe pattern is implemented by the inventory and notification service by handling the events published by the order service.

3.1.7 Event-Driven Service Topologies

As we discussed before, event-driven services typically react to events; the name kind of gives it away, doesn't it? However, architectures composed exclusively of microservices that interact through events are rare. In the real world, often there is a mix between synchronous and asynchronous requests. As we discussed, event-driven indeed isn't a silver bullet, and even in an event-driven architecture, there are simple use cases that don't benefit from the added complexity of asynchronous interactions. This section will detail the ordinary interactions between services we often find and implement.

Typical CRUD (create, read, update, and delete) microservices have synchronous interactions. Users and other applications interact with them through an API to access their data. For example, if a service exposed a REST API, we could do an HTTP POST to create a given resource and an HTTP GET to obtain it, as pictured in Figure 3-9.

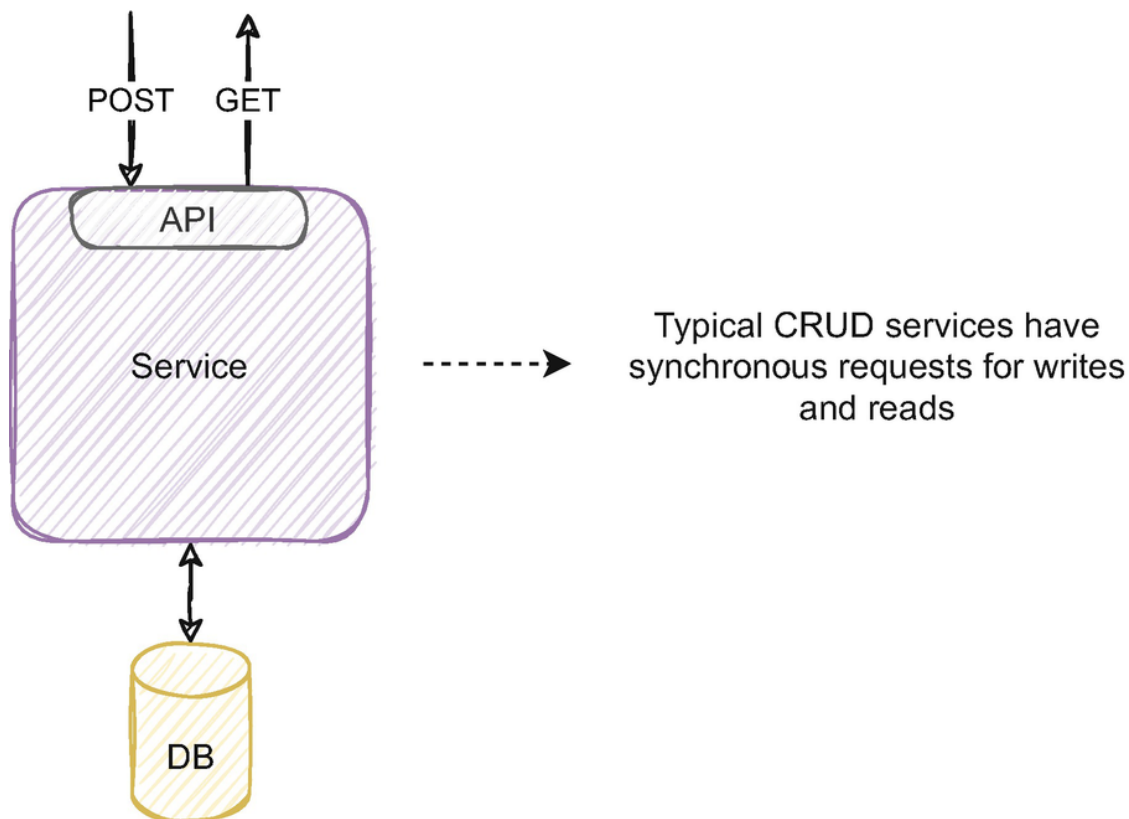


Figure 3-9 We can request and change information synchronously from CRUD services

An essential property of this topology is we obtain the response for our requests in a synchronous manner. If we create a resource, an order, for example, we immediately receive the response for the create request. If the service needs to validate the request to create the order, the service making the request obtains immediate feedback. Other asynchronous

topologies we discuss next might need more complex ways to give feedback to the service issuing the request. This topology is straightforward but lacks the advantages of event-driven services; it should be applied to simple low-scale use cases. Simple domains with low amounts of data and fewer dependencies from other services typically benefit from using this approach.

CRUD services can also publish events, as depicted in Figure 3-10. We can still evolve typical synchronous services to publish events to allow other services to react to them. Adding publishing capabilities in existing CRUD services can be a way to include existing services in a new event-driven architecture.

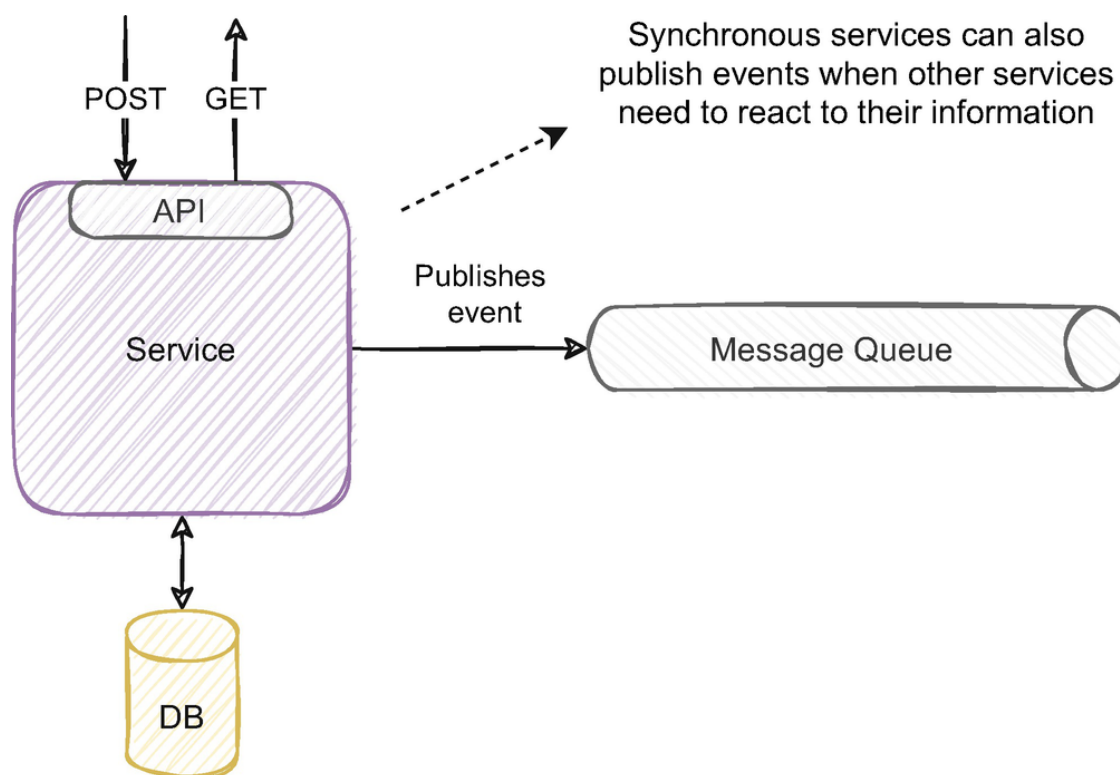


Figure 3-10 CRUD services publishing events when their data changes

By publishing the information to an event queue, we can benefit from the advantages of event streaming and share the data in the form of events to every other service that needs it. However, this topology and the one before suffer from the disadvantages we mentioned in Chapter 1. For example, suppose there are many services with synchronous APIs that call each other. In that case, we slowly but surely grow to a distributed monolith, where a failure in one service can cascade down to other services and affect the whole architecture.

As we mentioned in Subsection [3.1.2](#), commands are a type of message that request a given change to a service. Instead of receiving requests to change entities through a synchronous API like the examples we discussed before, we can receive through commands, as illustrated in Figure [3-11](#).

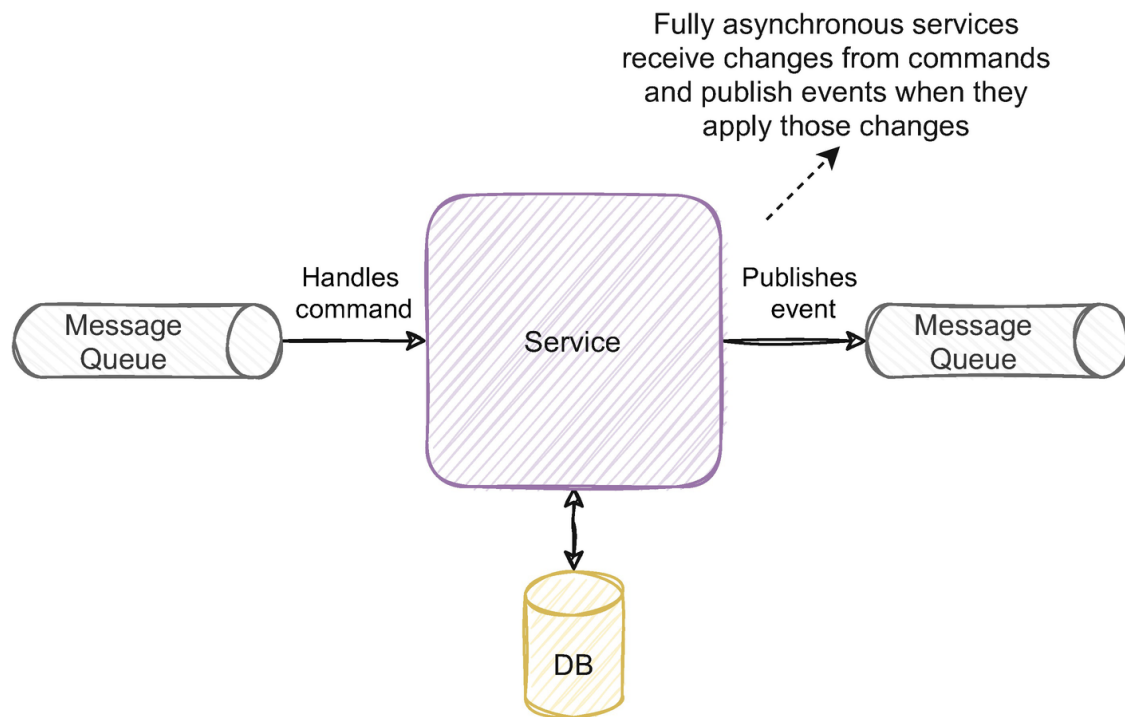


Figure 3-11 Services can be fully asynchronous and receive changes in the form of commands. Both the input and output are asynchronous

We send commands to a message queue, and like the example we discussed earlier about the order service, they request changes to a given entity. Once the changes are applied, the service publishes an event signaling those changes. This makes the service fully asynchronous and fully decoupled for other services. However, the fully asynchronous nature of the service can increase the complexity of different use cases that would be straightforward using a synchronous approach. For example, since we send each change in the form of a command to a message queue, there is no synchronous feedback to the user or system that triggered the request. If the command validation fails, giving feedback on why the verification failed can be tricky.

Services can also handle events and publish commands to other services, as illustrated in Figure [3-12](#). These services are typically called process managers or orchestrators. They react to information on different domains and instruct the services in its domain to react to those changes by sending commands. For example, in the order service, each order is an aggregate, and each command should only affect one

aggregate or entity. If a user changed its address on the user service, the order domain might need to reflect the change on all existing user's orders. To change each order, a process manager could react to the event and publish a command to each order.

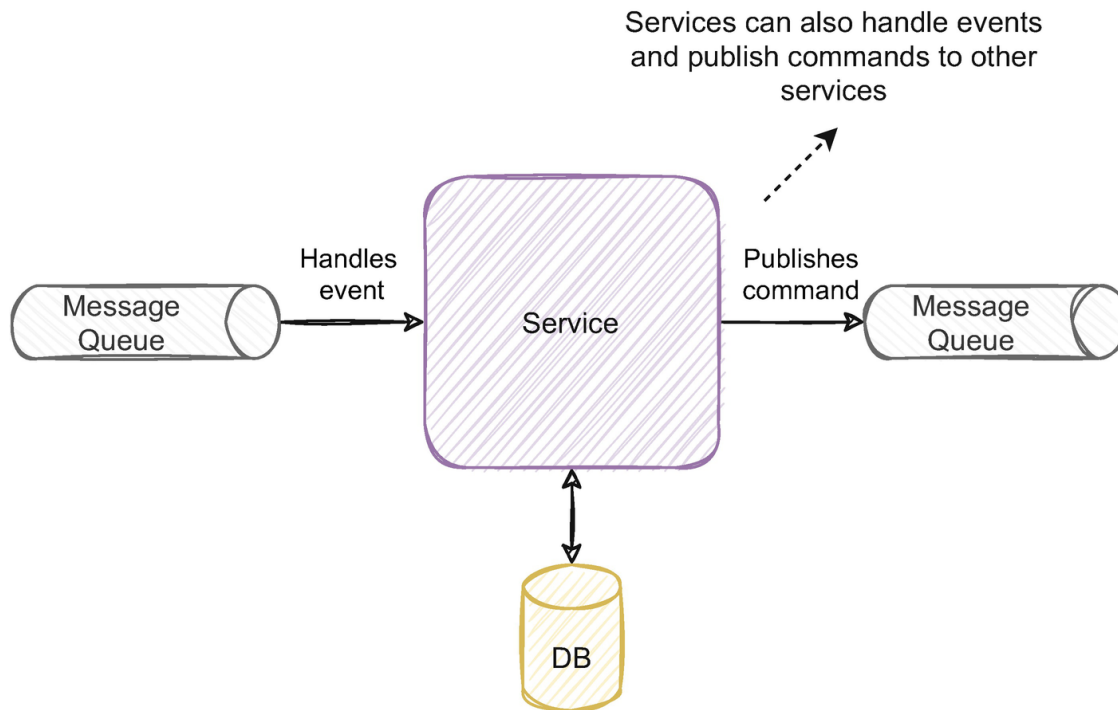


Figure 3-12 Some services can handle events and publish commands to other services

Instead of sending a command, this topology can also make a synchronous request to the target service depending on how the service exposes the functionality to change its resources.

The mix between these topologies and the synchronous and asynchronous interactions between the several services often raise difficult challenges in dealing with the eventual consistency associated with asynchronous services. Since some services might react to an event and facilitate APIs to expose their information, the request to obtain the information is synchronous, but the data is being changed asynchronously. The inherent eventual consistency can be troublesome to deal with, not only from a user perspective but also for other services consuming that information. How to deal with eventual consistency is further detailed in Chapter 5.

3.1.8 Common Event-Driven Pitfalls and Anti-patterns

Combining the patterns and topologies we just discussed sometimes originates in some common pitfalls. Most of them are very inconspicuous, and we might not notice they are there immediately, but they jeopardize the evolution of the architecture in the long run. In this subsection, we will discuss some pitfalls we often see people fall into.

Faking Synchronous Responses

As discussed in Chapter [1](#), an important consequence of event-driven architectures is the asynchronous nature they imbue in the systems. Often changes requested to the applications aren't reflected instantaneously, and feedback about the request often isn't returned in the same request flow. Often REST APIs, instead of returning the HTTP status codes of 200 (OK) or 201 (Created), can return 202 (Accepted). This status code means the system accepted the request; it was issued to be processed internally but wasn't yet processed. Often APIs convert requests to commands and send them to a queue; the response to that command is processed when the service handles that command. Since it is a queue, the time to respond can vary depending on the system's load; if there is a peak in load, the service might lag behind the number of messages being produced and might take a while to process it.

Since the response to the original request was accepted, retrieving the feedback to the issuing system or user might be problematic. For example, suppose you edit your billing address on an eCommerce platform, and after editing, the changes aren't reflected right away. In that case, it might be a bit annoying, but it probably isn't critical. However, in other use cases, it might be; if other services depend on that information, they might infer state based on stale information. In those cases, we might be tempted to fake synchronous responses by blocking the request and polling until the change is reflected.

Forging synchronous requests is an approach we often see, and although it gives a feel of being more consistent, it is very susceptible to issues when there is a higher load in the system. The minimal increase in message production might trigger timeouts in the blocked requests waiting for a response. Overall, it makes the system more brittle and susceptible to failure. If we use an event-driven approach, we should embrace the

asynchronous nature and the eventual consistency they imply and avoid faking guarantees that no longer exist. Chapter [5](#) details strategies to deal with these issues.

Command Publishing

As we mentioned in Subsection [3.1.2](#), commands target a specific service and a specific domain. Sometimes we might feel the need to publish a command to several different services. This often implies that either the command has too much responsibility or there isn't a clear domain separation between the services. Services should react to events, to something that already happened in a domain, and they need to do something about it. Several different services shouldn't handle a request to change a given entity. They should react to each other rather than receiving the same order.

For example, if we bought a product in an eCommerce platform, it would trigger a command to create an order. Still, only the order service would handle that command; the inventory and notification service wouldn't know the command. Neither should they because the order creation request might be rejected. The only service responsible for managing the order domain is the order service. The other services should only react to things that already happened in that domain, like the order created event.

Passive-Aggressive Events

Events inform that something changed in some service. A common pitfall is to imbue an event with a hidden responsibility to do something in another service (Martin Fowler mentions this pitfall in this [8](#) article). Much like when my wife states that there are dirty dishes in the sink, she is stating the fact that there are dirty dishes in the kitchen, much like an event, but it has a hidden message behind it – that I really should do the dishes.

This pitfall often happens when an event from a service needs a given action to happen in another unrelated service. The change is typically modeled as a command, but the system that receives it needs other services to perform the action and publishes an event requesting that

change. An event shouldn't request anything; it is a statement of a fact. To coordinate changes between several services, we should use different strategies. Although there is a gray area when designing events and guaranteeing we are not implementing passive-aggressive events, when we need to coordinate changes between several services, we should be concerned in ensuring the visibility and reliability of the flow of the messages and accomplishment of the overall process. To do that, we often need to implement a Saga or a process manager, which we will discuss further in Chapter [4](#).

3.2 Organizing Event-Driven Microservice Boundaries

Event-driven microservices are highly decoupled from each other, which enables changing a service without affecting other services. The ability to change and deploy changes independently is a pivotal characteristic to support a rapidly growing and evolving business. As the system grows, the complexity can hamper our efforts to achieve a highly evolutionary and decoupled architecture. Organizing and defining boundaries between the growing number of services becomes a main concern. This section will discuss the different approaches to defining microservice boundaries.

Chapter [1](#) discussed how event-driven architectures have a highly decoupled nature due to using events as a form of communication and as the source of truth. However, as the system grows and becomes more complex, we shouldn't take this characteristic for granted. Even when decoupled by a message queue, if we need to repeatedly add functionality that sprawls several services, they will often depend on each other's changes. This can easily lead to gridlock releases where several services need changes from other services to be released. They often need a strict order of releases and highlight complicated dependencies between a complex network of services. We can contain these dependencies if services have clear boundaries between them, much like the boundaries we create inside an application to separate the application logic and database logic.

There are several approaches to defining a microservice boundary. A good rule of thumb is code that changes together stays together. In Chapter [2](#), we deconstructed a monolith by using a domain-driven approach. We believe this is a sensible way to approach a boundary definition. Domains tend to be more stable than other types of approaches, and boundaries defined by having the company's domains in mind tend to accommodate changes without propagating them throughout the whole architecture. By drawing boundaries around a domain, it gives the space to evolve the domain and, at the same time, limit the impacts on other domains. They also tend to correlate well with the company's organizational composition; often teams are arranged around domain concepts, which translates well with an architecture with boundaries around those domains. Domain-driven design relates well with this kind of organization, and we deep dive into it in Section [3.3](#).

There are other approaches to organizing boundaries; you will often find that defining those boundaries and understanding where to fit new functionality are often a combination of those approaches. Keep in mind there isn't only one way to define something; a domain approach is a quite reasonable option, but it is important to understand it is not the only way, and often we benefit from combining more than one approach. In Chapter [2](#), we discussed we should have a strong reason to adopt an event-driven architecture, which can also play a role in this definition. Different reasons can drive different approaches and priorities.

3.2.1 Organizational Composition

Conway's law states that "Any organization that designs a system will inevitably produce a design whose structure is a copy of the organization's communication structure." If you look back at the architectures and organizational compositions of those architectures, you will probably find evidence of Conway's law even in awkward and funny examples. There was a company I worked with that had a boundary that, in the domain perspective, it made sense to be together with another existing boundary. No one could find a good name for it or to give a precise definition of why it was different from the existing one apart from small details. But why was it a distinct boundary? Well, it happened that the team that frequently

changed it was in a completely different organizational structure than the teams from the existing boundary. The company's organizational composition actually influenced the technical boundaries.

It isn't necessarily wrong; in that context, teams inside the same organizational composition that owned services inside the same boundary were more autonomous and facilitated feature implementation. However, it is essential to understand its implications and use them to our advantage. That might mean shifting teams to different structures to achieve the desired architecture. The inverse will inadvertently happen; the way teams are composed will invariably influence the system's boundaries.

If we align this concept with the domain approach in the case of an eCommerce platform, we might have a composition of teams that work with order management, another for product management, etc. This organization will also allow those teams to specialize in the domain concepts of the services they work with, which benefits the overall modeling between the conceptual and technical domains.

3.2.2 Likelihood of Changes

An approach to design boundaries is to separate services into two groups: services that change and are likely to change often and services that don't. This separation allows for a boundary where most of the business innovation happens and another for services that don't and won't need frequent changes.

Conceptually, it makes sense to group services that are likely to change together. If our objective to adopt an event-driven microservice architecture is to optimize time to market, it makes sense to group those services and adopt different approaches for each boundary. However, in practice, it is hard to foresee which services will belong to each group. Historically, we can perceive which services were often changed and which ones currently aren't. But understanding how they will evolve in the future usually has a high degree of uncertainty. Also, changing one service that suddenly needs frequent changes from one boundary to another is often la-

borious. All the assumptions we made until now were that the service would hardly change; changing those assumptions can be problematic. Usually, new functionality ends up affecting services that supposedly don't change that often, leading to even higher time overhead.

We did experience some successful use cases when combining this approach with the domain approach. Typically inside a domain boundary, some services change more often than others, and we can benefit from having those identified and specific approaches to deal with them.

3.2.3 Type of Data

Some services have to deal with sensitive data like PII (personally identifiable information), PCI DSS (payment card industry data security standard), or PHI (protected health information). This kind of data often has additional regulations and requires audits from external parties. Implementing these requirements (like GDPR) can add a significant effort (as we discussed in Subsection [3.1.1](#)), and we might benefit from limiting those changes to the services that actually handle that kind of data. Figure [3-13](#) illustrates several boundaries with different types of data.

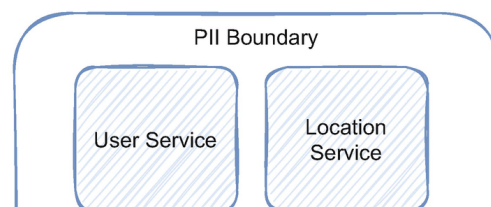


Figure 3-13 Example of defining boundaries by the type of data existing in the services

By having boundaries that handle the sensitive data and guaranteeing that the data only flows inside that boundary, we can focus the regulation's enforcement in a subset of the system. Some requirements, like the right to be forgotten and external audits to the system, can be hard to implement and manage. This type of organization enables to apply those requirements only to the services that need them. For example, inside the PII boundary, all services would have to apply the requirements, but the services in the non-sensitive data boundary wouldn't need to. This way, it is easier to manage and implement the requirements than addressing them in the whole architecture.

3.3 Brief and Practical Introduction to Domain-Driven Design and Bounded Contexts

Eric Evans first introduced DDD (domain-driven design) in his book⁹ about this subject. It is an approach to design complex systems that provides a set of techniques to help us understand the application's domain and build it into a conceptual model. The complete details of these techniques and ideas are outside of this book's scope, but we will approach how they are important and how we can use them in designing boundaries between event-driven microservices.

One challenge developers always struggle with is to capture reality and model it into code. DDD can help us to design the domains that the business comprehends in our applications. Models and boundaries designed around the domain are likely to be more stable and translate the business's reality more accurately. It also facilitates the composition of the company allowing it to organize teams around those business concepts.

One key aspect of DDD is the focus on the domain. The company's business value is in its domain; by focusing on it, we are guaranteeing we are modeling what really matters or what differentiates the business. Any system has to model some kind of business value into code. Often the translation from the conceptual domain into code gets lost in translation. By having close communication between the conceptual and coded world and sharing the same language, we greatly reduce the area of failure. DDD focuses on exactly that, having a close communication with the do-

main experts and together developing a conceptual model of the domain. DDD also introduces a concept of a common language used by both business experts and developers (named as ubiquitous language) to reduce the risk of misunderstandings. Using the same names the business uses in the code, we reduce the risk of unclear requirements and business rules.

DDD enables us to reason with a complex, intricate domain to conceptually divide it, allowing several teams to work on it sustainably. Having said that, simple straightforward systems aren't likely to benefit much from some of the patterns typically used with DDD, much like we mentioned in Chapter [1](#) about event-driven microservice architectures. DDD provides the tools to model and maintain a complex, long-lasting system. If our project is small or short-lived, we won't probably benefit from it. But it might benefit from its mindset.

Although the patterns associated with DDD like event sourcing or CQRS (we will detail these in Chapter [4](#)) aren't a one-fits-all solution, the value we extract with DDD and the mentality shift it inspires are very important. We might end up with a small, simple system, but whichever system we are doing, having a clear understanding of the domain is pivotal to its success. Having a close communication with domain experts and having a clear understanding of the domain and how it maps to our implementations enable us to focus on the domain, where the real value is. A design approached this way enforces the importance of understanding and modeling the company's business value.

3.3.1 How We Can Apply It in Practice

Domain models are the core of the design. These models have detailed business knowledge and capture the domain knowledge from the domain experts. These models are converted to code and can scope the boundaries of that knowledge and verify its integrity. The teams designing the models have close communication with the domain experts, and they work together to develop these models. As the business evolves, so do the domain models.

By developing the domain models, we can understand the existing domain and its interactions. We lightly touched this subject in Chapter [2](#) when we discussed how to

move from a monolith to an event-driven architecture. The exercise in understanding the monolith's existing domain is the same exercise to design the domain models, and we might end up with something similar to what we discussed in Chapter 2, illustrated in Figure 3-14.

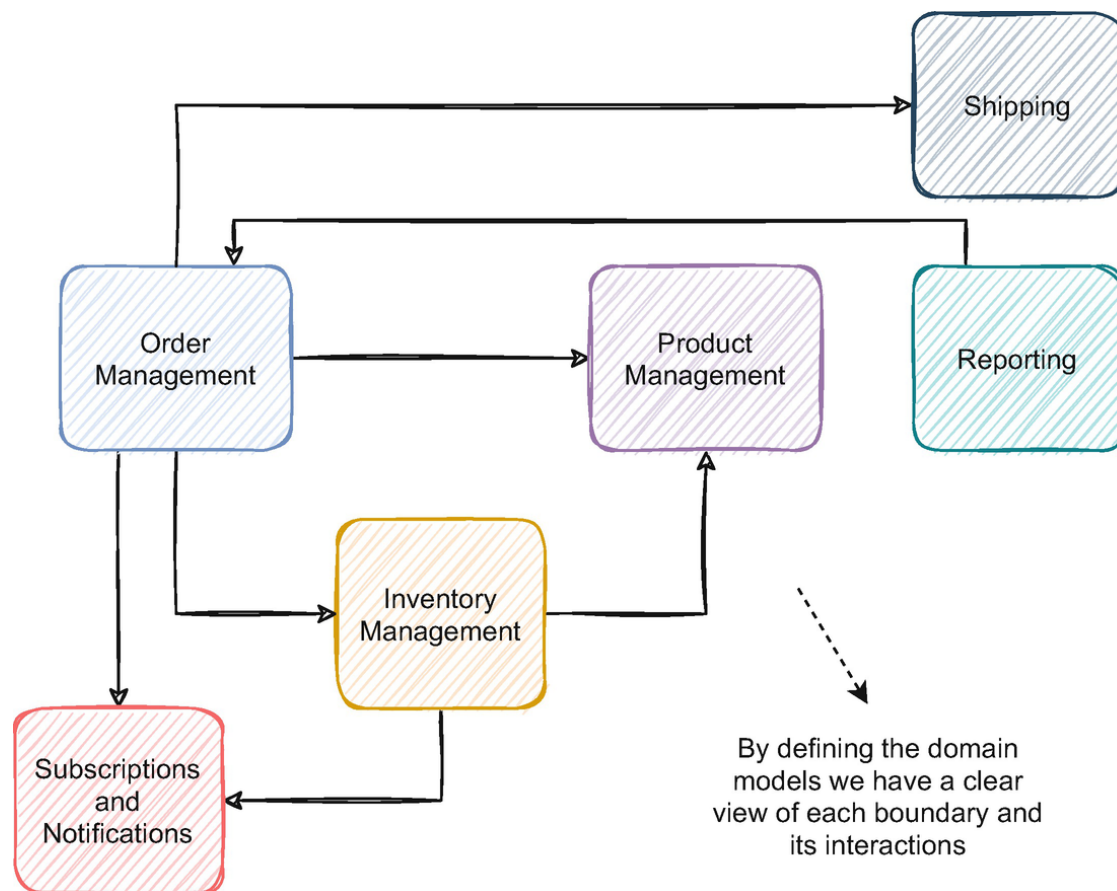


Figure 3-14 The same domain models we discussed in Chapter 1. By defining them, we can have a clear understanding of each bounded context and its interactions

When analyzing our existing eCommerce platform, we defined several different domains illustrated in Figure 3-14. Each of them is what DDD defines as a bounded context, and they usually represent a more extensive business domain concept. This domain concept often translates to a boundary and abstracts those domain details from the other bounded contexts. For example, the shipping bounded context might need to know the different shipping providers, but the order management bounded context doesn't or shouldn't.

Bounded contexts have one or more aggregates. There are many definitions of aggregate, but they often represent conceptually a domain unit inside the boundary. For example, in the order management bounded context, we could define the order aggregate, the product aggregate, etc.

The aggregate models a domain concept, and an entity is an instantiation of that concept. We could define the order aggregate, and the specific order 231 is an entity.

Typically, services request changes to entities through commands, and changes are propagated through events. Aggregates can reject requests to changes and are used to maintain the consistency and to enforce the business rules related to that aggregate. Aggregates can also be exposed to other bounded contexts or can be hidden depending on the domain. For example, the order management bounded context might have an order and product aggregate. It might make sense to expose the order aggregate in the order bounded context since it is the core concept in that bounded context. The product aggregate, however, might be needed just as a reference to another domain (the product bounded context), and since it is only an internal view of that boundary, it might not be exposed. Figure 3-15 illustrates an example of an aggregate composition in the order service.

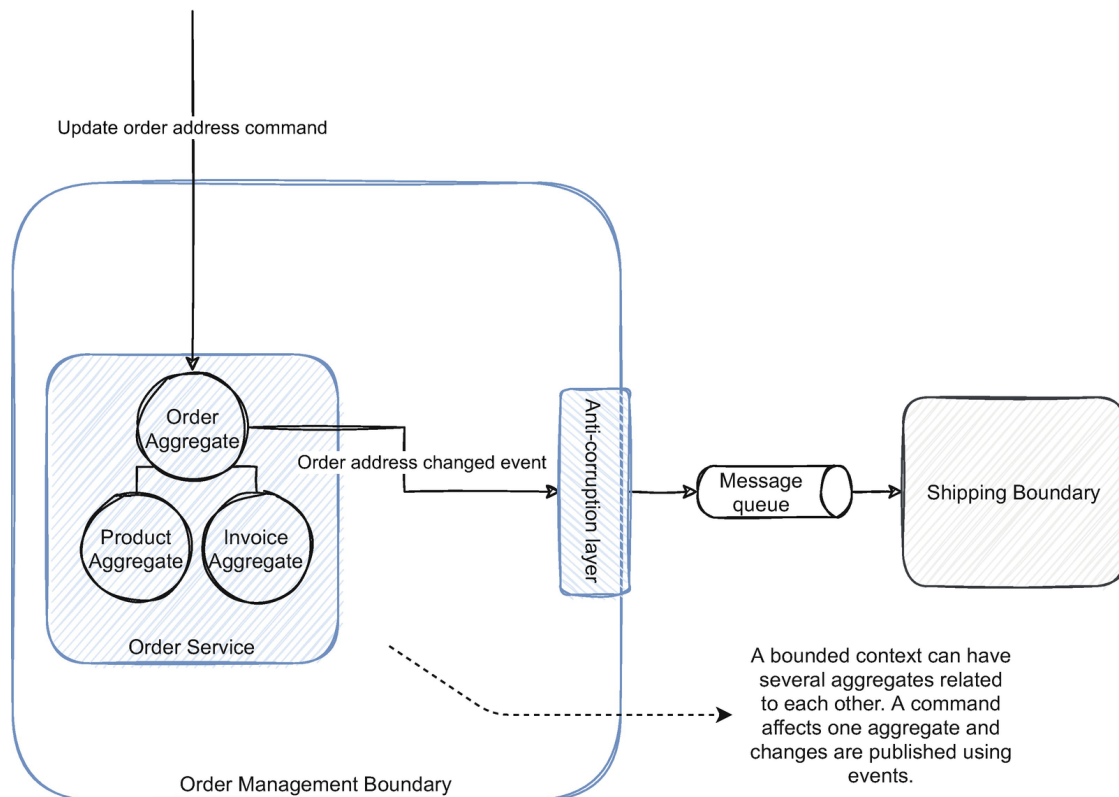


Figure 3-15 Example of an aggregate composition for the order service. The order aggregate has two other associated aggregates

We could define an order aggregate that relates to two other aggregates, the product and invoice aggregate. To change an order, we can issue a command to the order aggregate affecting a single entity (or a specific existing order). The command to change the order address can be rejected depending on the business rules the order service enforces. We can

communicate changes to aggregates to the other boundaries through events. Changing the order address triggers an order address changed event, which can be consumed by other boundaries to apply their own domain logic.

Each bounded context has its domain model. Since bounded contexts have to communicate with each other, it is essential to guarantee that the domain concepts from one bounded context don't leak into another. The anti-corruption layer enforces that all domain-specific logic and information remain inside the boundary.

A boundary can be implemented with a single service or can be composed of several different microservices. Figure 3-15 shows that the order service and the anti-corruption layer can be two different independent services or one single service. A piece of reasonable advice is to take the same approach we discussed in Chapter 2; first create arguably extensive services that represent a complete boundary. Later, if necessary, we can decompose that service into smaller parts. In more complex architectures, boundaries are composed of several services that work together to accomplish that domain's purpose and then communicate the changes to that domain to other boundaries that are also composed of several services. A reasonable approach is to do that incrementally; first we can understand the existing domain model, then create a single service to implement that bounded context, and then decompose that single more extensive service into smaller parts if needed.

Aggregates usually have a strong relation to concurrency and atomicity. Typically, changes to an entity are performed without concurrency or in single thread. That doesn't mean the service is single-threaded, though; the service can change multiple orders simultaneously as long as the changes are to different orders. If we change three different orders simultaneously, those changes can happen concurrently, but if we change three times the same order, those three changes typically occur sequentially. This concurrency management guarantees each aggregate's consistency while allowing parallel processing (how to handle concurrency will be discussed in Chapter 6).

The aggregate decomposition of a bounded context can be highly subjective. In Figure [3-14](#), we designed three aggregates, but we could quickly come up with a different design. As we will discuss in Section [3.4](#), aggregate design can have a considerable impact in the system and its performance. An arguably good approach is first to understand the domain conceptually and work closely with the domain experts to design a mental model you both agree with. Then map that model to the technical implementation and understand how the technical and performance implications influence the model.

3.4 Event-Driven Microservices: The Impact of Aggregate Size and Common Pitfalls

In Section [3.3](#), we discussed how to use DDD to understand and design an application's domain and boundaries. We discussed how the aggregate definition was a pivotal step in that design and that it can have a high impact on the whole system. This section will detail how it can affect the whole architecture's performance and how to avoid some common pitfalls.

We discussed that entities are an instantiation of an aggregate; for example, for the order aggregate, a possible entity could be the order with id 231 submitted by user John. We also discussed that aggregates have a strong relation with concurrency since each entity is changed transactionally (when someone is changing an entity, no other changes can occur simultaneously). Thus, deciding the correct granularity of the aggregate becomes pivotal and relates closely to the ability to guarantee consistency and performance in the system.

Let's illustrate this with an example; imagine we work with an eCommerce platform that sells beauty products. Each product can have several variants; for example, perfumes can have different bottle sizes, lipsticks can have different colors, etc. The stock is managed at the variant level; we could have in stock one small bottle of a given perfume and two large bottles. Products have a precise categorization, and the business often applies changes to categories of products. For example, from time to time, they might want every lipstick to have a given discount, or they might

want to block the perfume category to ship to some countries due to the shipping laws of those countries, etc.

Typically, changes are applied to a single entity; changes that affect several entities or several aggregates often need the coordination of a Saga pattern or process managers (we discuss these patterns in Chapter [4](#)). Once something is changed in a given entity, an event is published informing that change; thus, domain events usually have the same granularity of the aggregate definition.

There might be three different aggregates we could define: category, product, and variant. Using the variant as the aggregate would facilitate the stock changes and enable a high concurrency system to guarantee performance (since changes could be made to several variants in parallel). But it would also publish several smaller events; depending on the number of products and variants, this could mean a lot of stress on the message broker and the consuming services. While defining the category as the aggregate would guarantee the consistency of the larger discount and visibility changes and would publish larger individual events. But it could easily undermine the system's performance since each product in each category would be changed sequentially.

An arguably good first step is to define the aggregates conceptually, understanding the domain and designing it having in mind a real domain concept. But we shouldn't fall into the trap of creating using only the context of our service or boundary. For example, if we were in charge of developing the stock management service, a good aggregate would be the variant aggregate. The same could be said if we were in charge of designing the system to manage the product's categorization; it would certainly make sense to us to define the category as the aggregate.

However, a second good step is to understand who the consumers of those events are and the reality of those services and boundaries. What does their aggregate look like? Is it roughly the same as ours? Is it a larger domain concept or a smaller one? For example, let's say the service managing stock had a variant aggregate and published an event for each variant. The service applying the discount would need to know which variants have stock but had to apply the discount to a category of products.

That could comprise several hundreds of products, each with several variants. The mismatch between the aggregates would mean that the discount system would have substantially more work to understand how each small change applied to their larger concept. Defining an aggregate as large as a category in the stock boundary might not make sense either. Still, perhaps if both boundaries adopt a product aggregate, they would benefit from the simpler communication between each other. Overall, understanding our consumers' needs and finding a compromise are often best for the overall architecture.

Defining the size of aggregates is always subjective. Changing it later on can also be a laborious task. However, when in doubt, rely on your domain experts' support and your understanding of the domain. Asking these questions often helps to decide:

- Does the aggregate translate to a real domain concept?
- How many events will we publish, and how granular are they? Can the message broker and the consuming services handle the load? Are we publishing an unnecessarily large quantity of events?
- How performant and scalable the system can be? If we grow to the millions of aggregates, does the system remain performant? Can we change several aggregates at the same time and scale seamlessly?

Sometimes you will find you got the aggregate size wrong. Often it is hard to foresee the load and usage of the system accurately. Don't be afraid to advance and change later on. Although these changes can be laborious, as we discussed in Chapter [1](#), the decoupled nature of event-driven services allows us to replace services reasonably easily. We will further discuss the impact of the event schema's size in Chapter [8](#).

3.5 Request-Driven vs. Event-Driven Services

In Subsection [3.1.5](#), we discussed several service topologies. When building a service in an event-driven architecture, the two main approaches are using a request-driven or event-driven service. Obviously, event-driven architectures are composed of event-driven services. However, most architectures aren't composed of only event-driven services. They arguably shouldn't be since the two approaches have different implications

and aren't adequate for all use cases. In this section, we will discuss the two approaches and guidelines to which one to choose.

Event-driven architectures are characterized by using events as the source of truth. As we discussed in Chapter [1](#), this enables several powerful use cases, and we should follow this guideline throughout the architecture. Traditional microservices typically expose synchronous APIs and use them to provide access to their data. These kinds of services are simpler to use and to reason with, and often there are use cases where an event-driven approach to a microservice simply isn't the best way to go. Figure [3-16](#) illustrates the typical approaches of request-driven and event-driven services.



Figure 3-16 Differences between the communication of typical request-driven and event-driven services

A request-driven service typically exposes a synchronous API like REST to expose functionality to other applications. We can access data by issuing requests to the same API. Functionality exposed by request-driven services is easy to reason with; it is similar to a call to a local function we would do in the code, but instead of running in memory, it does a remote network request.

Request-driven services have a place in event-driven architectures; not all use cases are suitable for event-driven services. Most often than not, you will find and work with both types of services in the microservice ecosystem. Event-driven services introduce an asynchronous nature to the interaction between the applications. Some use cases don't benefit from the additional complexity or cannot afford the eventual consistency introduced by event-driven services. Stronger consistency guarantees are hard (or impossible) to maintain in distributed event-driven microservices. If there is a use case for strong consistency, choosing a synchronous interaction in a request-driven service is often a better choice than using an event-driven one.

There are several strategies to deal with eventual consistency, and eventual consistency doesn't need to be slow (we further detail both points in [Chapter 5](#)). Still, there might be use cases where we simply can't afford any inconsistency whatsoever. Don't misunderstand me; we always need consistency, but it's relatively debatable whether we can maintain distributed strong consistency at scale (thus the rise of NoSQL databases).

We did come across some use cases where strong, consistent, and synchronous responses were needed in low-scale services. In these use cases, a synchronous request-driven approach is often a better choice due to the simplicity of accomplishing the use case. For example, if we have a front-end application that makes a couple of hundreds of requests per day to a service and needs a synchronous response, we probably wouldn't benefit much from the added complexity of asynchronous events between the two applications. We can also publish events in request-driven services to provide an event-driven approach to share their data.

However, we should avoid the pitfall of using a high quantity of request-driven services throughout the whole architecture. One important consequence is turning our microservice architecture into a distributed monolith. Frequent requests between the services often lead to high coupling between the services, which impacts new features and leads to complex dependencies. We also discussed in Chapter [1](#) how request-driven services are susceptible to cascading failures. A failure in one apparently not critical service can often bring the whole architecture down by cascading the failure through the complex network of synchronous requests. When the architecture starts to grow, it's often hard to understand the advantages of a complex network of request-driven services and a monolith.

Sharing data through synchronous APIs can be troublesome when services need a huge amount of data or frequent lookups. These kinds of requests often impact the service and limit their resources. Requesting a large chunk of a service's data can significantly impact the database, and doing so through regular paging often has bad performance. Those use cases really shine when streaming data through an event stream. As we discussed in Chapter [2](#), using event streams with event-driven services provides a very organic way to share the data without impacting the service that owns the data.

Be sure to focus on the event stream being the source of truth and the medium services share data. Using request-driven services in specific use cases makes sense, and we shouldn't be dogmatic on whether choosing only one type of approach.

3.6 Deciding When to Create a New Microservice or Add Functionality to an Existing One

Once we have an event-driven microservice architecture up and running, where to incorporate new functionality can be dubious. There are usually two options: either add new functionality to an existing service or create a new service. The decision often depends on the functionality being added and the existing architecture. There isn't a one-fits-all rule, but a few pointers can help us decide which one to choose.

We can use DDD to map the different bounded contexts in an existing system, but migrating existing functionality to new services by either creating a new service or adding it to an existing one can be subjective. For example, suppose a part of our system manages orders and deals with orders and order lines. In that case, an arguably clear decision is to maintain both of them in the same service due to the close relationship between the two entities. However, if we also manage stores and each store has stock, should we create a separate service for each concept or maintain both in the same service? And how about adding new functionality? If we received a requirement to add several complex pricing rules to our system, should we add those rules to the existing product system or create a separate system to manage those rules? The answer is subjective, and usually there isn't a right or wrong answer, just tradeoffs between the two solutions.

As discussed in Chapter [2](#), a sensible approach is to create coarse-grained services and then fine-grained services as needed. It's better to incrementally evolve and add complexity to the architecture than starting with several dozens of services early on. However, suppose our services begin to get too large and suffer from some of the limits of monoliths we discussed. In that case, it might be advisable to review the service and understand if it is handling too much responsibility. Either way, don't be afraid to try and adapt later; although it might lead to a lot of rework, the high decoupling of event-driven architectures allows us to swap services as long as we don't break the event contracts.

Here are some considerations we can keep in mind to help us decide. These points can help you sway in one solution or another, considering both can help you shift toward the best decision based on the system's characteristics you want to prioritize.

Some points that indicate we should add the functionality to an existing service:

- There is a very close relationship between the conceptual domain of the new functionality and an existing service. In the example we discussed about the order and order lines, they are very related domain concepts, and in many systems, it might make sense to keep them together. Typically, we want code that changes together to be together.

So domains that are closely related and are likely to change together, keeping them in the same place might be a sensible decision.

- The new functionality will need to share another existing service's database. We discussed the disadvantages of sharing a database in Chapter 1. Handling schema changes between several services can make new functionality difficult to add and produce complicated lock-step releases. Also, different services that use the same database are keen to impact each other, for example, with a long-running transaction, which turns scaling each service into a challenge. If we need to add a new service that will need to share the same database as another one, it is a giant red flag, and we should avoid it at all costs. We can still create a new service if it needs a large amount of data from an existing service, but by consuming the event stream and creating an internal view of the data. Adding functionality to an existing service or using the event stream is almost always a better option than sharing a database.
- The anemic domain model 10 is a common object-oriented anti-pattern and a common pitfall in many services. When building a microservice architecture, you might find anemic domain services. These services are characterized by a simple sheath around the database that only writes or fetches information, lacking domain logic inside the service. Basically, it looks like a repository pattern with associated hardware. If the service lacks domain logic, odds are it is distributed somewhere else in the system. This characteristic can be a good sign to add functionality to that service and enrich it with domain value. It's not a rule though; some services aren't heavy on domain logic; they can perform more mundane tasks or are endowed with a more infrastructural nature.

Some points that indicate the functionality might belong in a new service:

- If we add functionality to a service that depends on multiple services, it might be a sign that this service has a lot of responsibility. In this case, we might benefit from creating a new service and even migrating some functionality from the existing service. The service can consume events from a myriad of different sources or request information from several services; either way, it might indicate we centralized

a large quantity of logic in that service. Increasing its complexity might not be advisable and can be a sign we should separate some of that logic.

- Several different teams manage the existing service, and there is no clear ownership. An advantage of a microservice architecture is having autonomous teams who can deliver fast with minimal intervention from other teams. If we add functionality to a service already managed by several teams, it might be advisable to understand how we can divide the service and its responsibilities. Just this argument by itself might not be a strong enough reason to create a new service since we can manage this in other ways, but combined with others, it might form a strong argument.
- The new functionality requires a different technology stack. If we are adding a new functionality that we want to use a different technology stack or requires specific technology, adding it to an existing service can be challenging. Managing two different technology stacks in two different services makes the maintenance, release, and deployment a lot easier.
- When a new functionality has a very different usage pattern and needs to scale independently of any other service, it can be a good sign to separate it into a new service. Suppose we add functionality that will have several times of magnitude more usage than the current usage. In that case, it might mean we will need more resources for that functionality than the existing ones. Another use case is if the usage has seasonality associated with it; for example, every Friday has a large peak of usage. It might be effective to scale only that part of the system independently of the rest of the system. We can easily achieve that if the functionality is in a separate service.

It's fairly apparent where we should add most new functionality. However, from time to time, the question to create a new service or use an existing one arises. It is important to understand the implications and be able to build a strong argument. Each of these arguments alone might not mean much; the goal here is to give us the tools to build a strong, compelling case for either decision.

3.7 Summary

- Services can and should have the code organized in a tiered architecture. The clean architecture is a good example of an architecture we can use that promotes decoupling and enables easy maintenance.
- As the architecture grows to more fine-grained services, each service can translate to a different clean architecture's layer.
- Although we can use either durable or ephemeral message brokers, they accomplish different purposes. Event-driven architectures typically have high synergy with durable message brokers.
- Complying with some data security standards can be challenging in event-driven architectures. Crypto shredding is an option we can use to leverage those requirements.
- There are typically four types of entities in an event-driven architecture: commands, events, documents, and queries.
- Using documents or events can be debatable. When a system needs a large part of the entity's data, a document is often a reasonable choice. If a system needs the meaning and the inherent change behind the event, partial events are often a better choice.
- There are typically three types of message patterns we can use: send/receive, publish/subscribe, and request/response.
- Faking synchronous response, command publishing, and passive-aggressive events are three common pitfalls we should avoid while designing event-driven systems.
- To design a system's boundaries, we can use four approaches: organizational composition, likelihood of changes, type of data, and domain-driven design. Although a domain approach is a sensible option, we often benefit from combining more than one approach.
- We can use domain-driven design to understand the application's domain and build it into a conceptual model. Models and boundaries designed around the domain are likely to be more stable and translate the business's reality more accurately.
- The aggregate size can have high implications in the system's consistency management and performance. There are a few questions we can ask to help us design those aggregates.
- We can use both request-driven or event-driven services. Although event-driven is our go-to approach, we can benefit from request-driven services in specific use cases. We shouldn't be dogmatic about choosing only one type of approach.

- There are several characteristics we can go through to help us decide to either add new functionality in an existing service or a new one.

Footnotes

- ¹ See Robert C. Martin, “The Clean Architecture,” August 13, 2012, <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
 - ² See “SOLID,” <https://en.wikipedia.org/wiki/SOLID>
 - ³ Further details in “Hexagonal architecture (software),” [https://en.wikipedia.org/wiki/Hexagonal_architecture_\(software\)](https://en.wikipedia.org/wiki/Hexagonal_architecture_(software))
 - ⁴ Further details in Jeffrey Palermo, “The Onion Architecture: part 1,” July 29, 2008, <https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/>
 - ⁵ See Brian McClain, “Understanding the Differences Between RabbitMQ vs Kafka,” November 16, 2020, <https://tanzu.vmware.com/developer/blog/understanding-the-differences-between-rabbitmq-vs-kafka/>
 - ⁶ See “Benchmarking Apache Kafka, Apache Pulsar, and RabbitMQ: Which is the Fastest?,” August 21, 2020, www.confluent.io/blog/kafka-fastest-messaging-system/
 - ⁷ See David Jacot, “Handling GDPR with Apache Kafka: How to Comply Without Freaking Out?,” May 13/14, 2019, www.confluent.io/kafka-summit-lon19/handling-gdpr-apache-kafka-comply-freaking-out/
 - ⁸ See Martin Fowler, “What do you mean by “Event-Driven”?”, February 7, 2017, <https://martinfowler.com/articles/201701-event-driven.html>
 - ⁹ See Eric Evans, “Domain-Driven Design: Tackling Complexity in the Heart of Software,” August 20, 2003, www.amazon.com/Domain-Driven-Design-Tackling-Complexity-Software/dp/0321125215
 - ¹⁰ More details in Martin Fowler, “AnemicDomainModel,” November 25, 2003, www.martinfowler.com/bliki/AnemicDomainModel.html
-

[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)